

Ingeniería de Software II

2026 – CLASE 9

Contenidos

- ❖ Estrategias de Prueba
- ❖ Depuración de programas

Enfoque estratégico de pruebas

- ❖ Una estrategia de pruebas del software proporciona una guía que describe los pasos a seguir, cuándo se planean y llevan a cabo, cuánto esfuerzo, tiempo y recurso se requerirán.



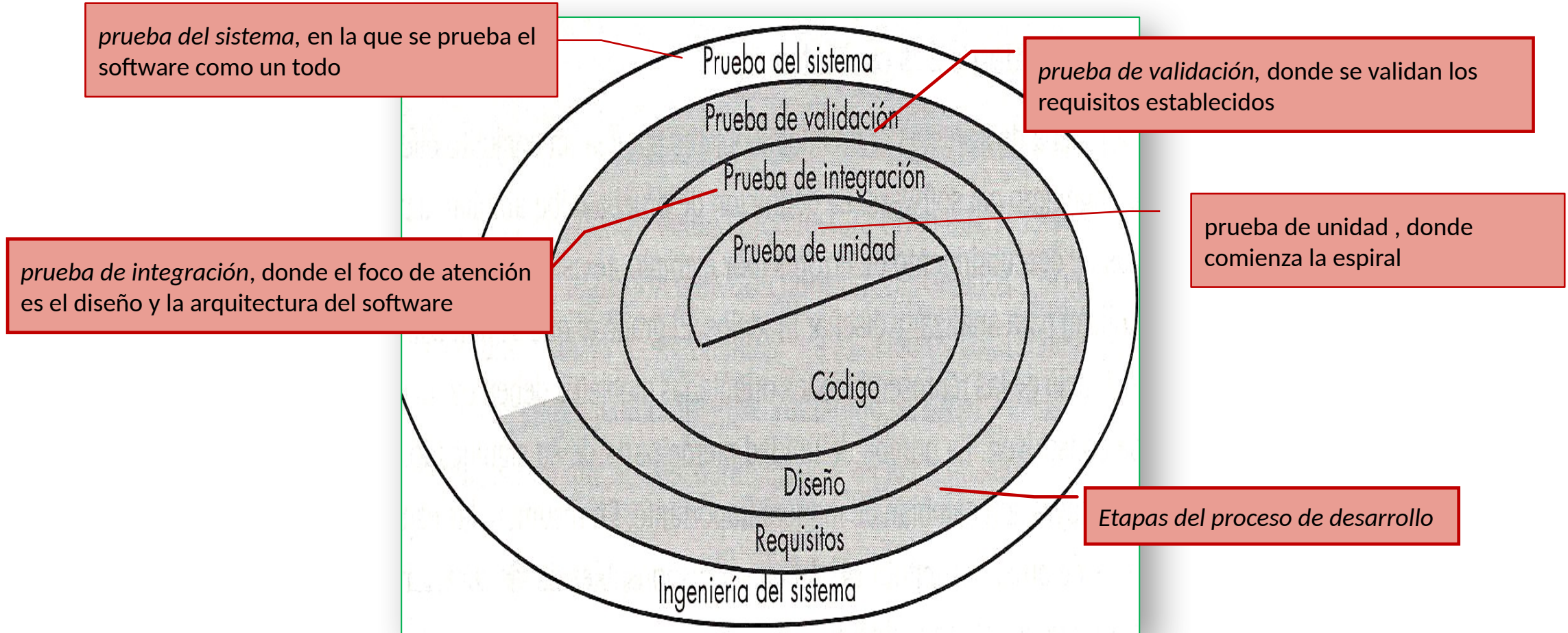
Enfoque estratégico de pruebas

- ❖ La prueba es un conjunto de actividades que se planean con anticipación y se realizan de manera sistemática.
- ❖ Conjunto de pasos en el que se incluyen técnicas y métodos específicos de diseño de casos de prueba.
- ❖ Una estrategia de pruebas debe incluir pruebas de bajo nivel y de alto nivel.
- ❖ Las actividades de las estrategias de pruebas son parte de la Verificación y Validación, incluidas en el aseguramiento de la calidad del software.

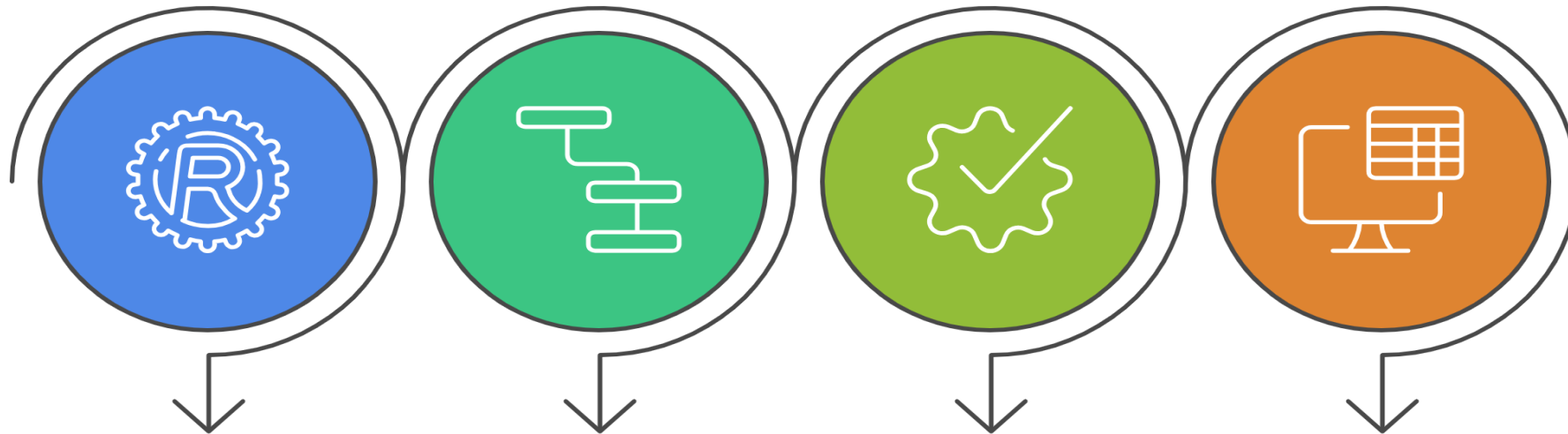
Concepto de Verificación y Validación

- ❖ La **Verificación** es el conjunto de actividades que asegura que el software implemente correctamente una función específica y la **Validación** es un conjunto diferente de actividades que aseguran que el software construido corresponde con los requisitos del cliente.
- ❖ **Verificación:** *¿Estamos construyendo el producto correctamente?*
Comprobar que el software está de acuerdo con su especificación, donde se debe comprobar que satisface tanto los requerimientos funcionales como los no funcionales.
- ❖ **Validación:** *¿Estamos construyendo el producto correcto?*
Es un proceso más general, cuyo objetivo es asegurar que el software satisface las expectativas del cliente.

Estrategias de prueba



Tipos de prueba de software



Pruebas de unidad

El componente verifica el correcto funcionamiento del componente

Pruebas de integración

Verificación de que los componentes trabajan juntos correctamente

Pruebas de validación

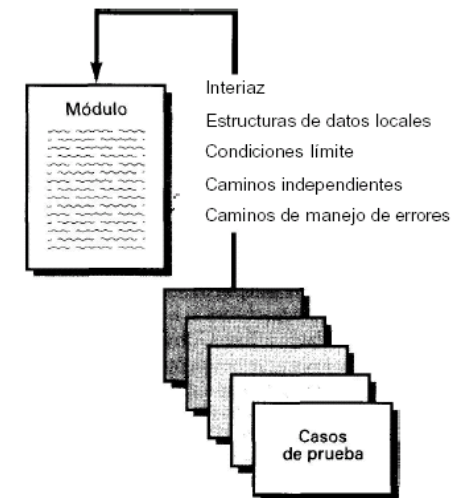
El software satisface los requisitos funcionales y no funcionales

Prueba del sistema

Verificación de que cada elemento encaja de forma adecuada

Pruebas de Unidad

- ❖ Se prueba la interfaz del módulo para asegurar que la información fluye de forma adecuada.
- ❖ Se examinan las estructuras de datos locales.
- ❖ Se prueban las condiciones límite para asegurar que el módulo funciona correctamente.
- ❖ Se ejercitan todos los caminos independientes.



Pruebas de Unidad

- ❖ Los errores más comunes detectados por las pruebas de unidad son:
 - » Cálculos incorrectos
 - Aplicación incorrecta de predecesores aritméticos
 - Operaciones mezcladas
 - Inicialización incorrecta
 - Falta de precisión
 - Representación simbólica incorrecta
 - » Comparaciones erróneas
 - » Flujos de control inapropiados

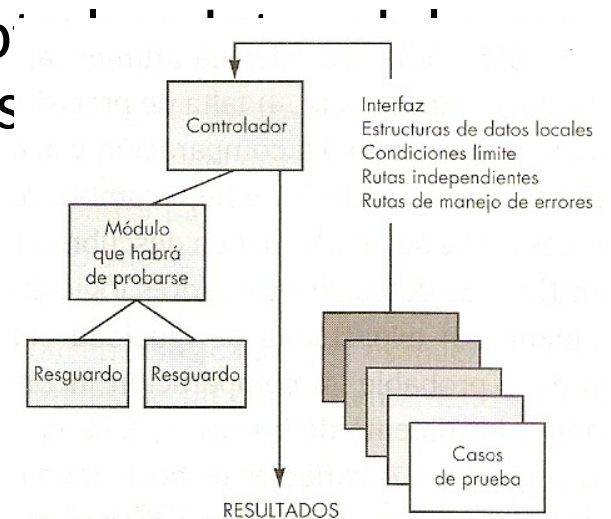
Pruebas de Unidad

- ❖ Los Casos de prueba deben descubrir errores como:
 - » Comparaciones entre diferentes tipos de datos
 - » Operadores lógicos aplicados incorrectamente
 - » Expectativas de igualdad con grado de precisión
 - » Comparación incorrecta de variables
 - » Terminación inapropiada o inexistente de bucles
 - » Falla en la salida cuando se encuentre una iteración divergente
 - » Variables de bucle modificadas inapropiadamente

Pruebas de Unidad

❖ Procedimiento:

- » Como un componente no es un programa independiente, se debe desarrollar para cada prueba de unidad un software que controle y/o resguarde.
- » Un controlador es un «programa principal» que acepta caso de prueba, pasa estos datos al módulo (a s) y muestra los resultados.



Pruebas de Unidad

❖ Procedimiento:

- » Un resguardo sirve para reemplazar a módulos subordinados al componente que hay que probar.
- » Los controladores y resguardos son una sobrecarga de trabajo.
- » Si los controladores y resguardos son sencillos, el trabajo adicional es relativamente pequeño.
- » La prueba de unidad se simplifica cuando se diseña un módulo con un alto grado de cohesión.

Pruebas de Integración

- ❖ Se toman los componentes que han pasado las pruebas de unidad y se los combina según el diseño establecido.
- ❖ En esta combinación es posible que:
 - » Los datos se pueden perder en una interfaz.
 - » Un módulo puede tener un efecto adverso e inadvertido sobre otro.
 - » La combinación de subfunciones no produzca el resultado esperado.

Pruebas de Integración

- ❖ El programa se construye y se prueba en pequeños segmentos en los que los errores son más fáciles de aislar y de corregir.
- ❖ La integración puede ser:
 - » Descendente
 - » Ascendente

Pruebas de Integración

❖ Integración descendente

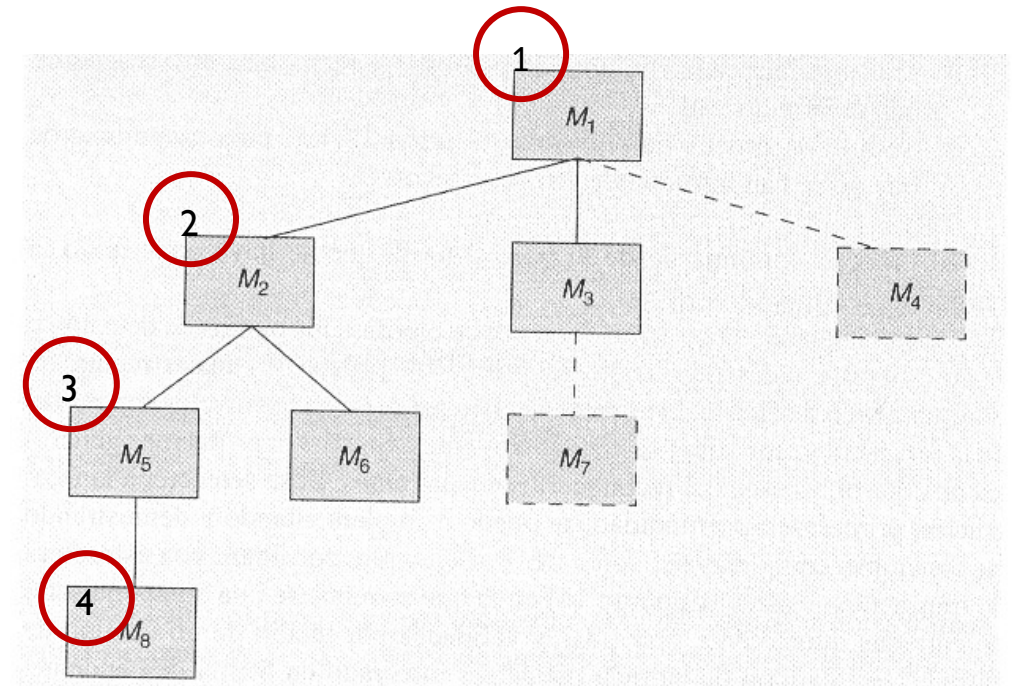
- » Los módulos se integran al descender por la jerarquía de control, iniciando por el programa principal.
- » Se puede realizar:
 - En profundidad: *integra todos los módulos de un camino de control principal de la estructura.*
 - En anchura: *incorpora todos los módulos directamente subordinados a cada nivel.*

Pruebas de Integración

❖ Integración descendente en profundidad

Pasos:

1. Conductor: Módulo principal
Resguardos: Para los módulos subordinados
2. Sustituir resguardos por módulos 1 a 1
3. Probar
4. Reemplazar otro resguardo
5. Pruebas de regresión

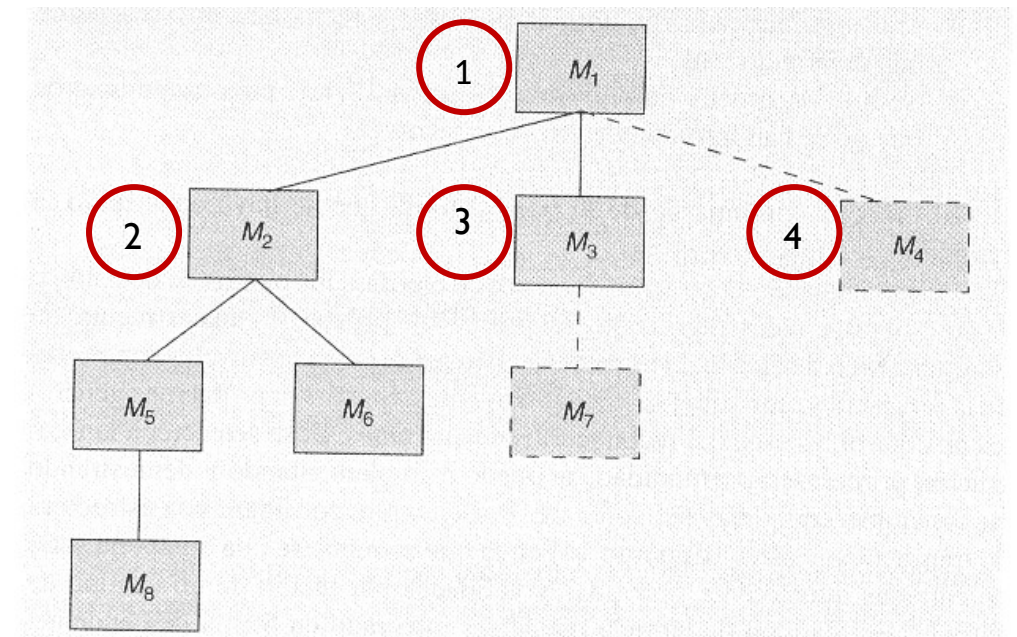


Pruebas de Integración

❖ Integración descendente en anchura

Pasos:

1. Conductor: Módulo principal
Resguardos: Para los módulos subordinados
2. Sustituir resguardos por módulos 1 a 1
3. Probar
4. Reemplazar otro resguardo
5. Pruebas de regresión



Pruebas de Integración

❖ Integración ascendente



Se empieza la prueba con los módulos atómicos (es decir, módulos de los niveles más bajos de la estructura del programa).



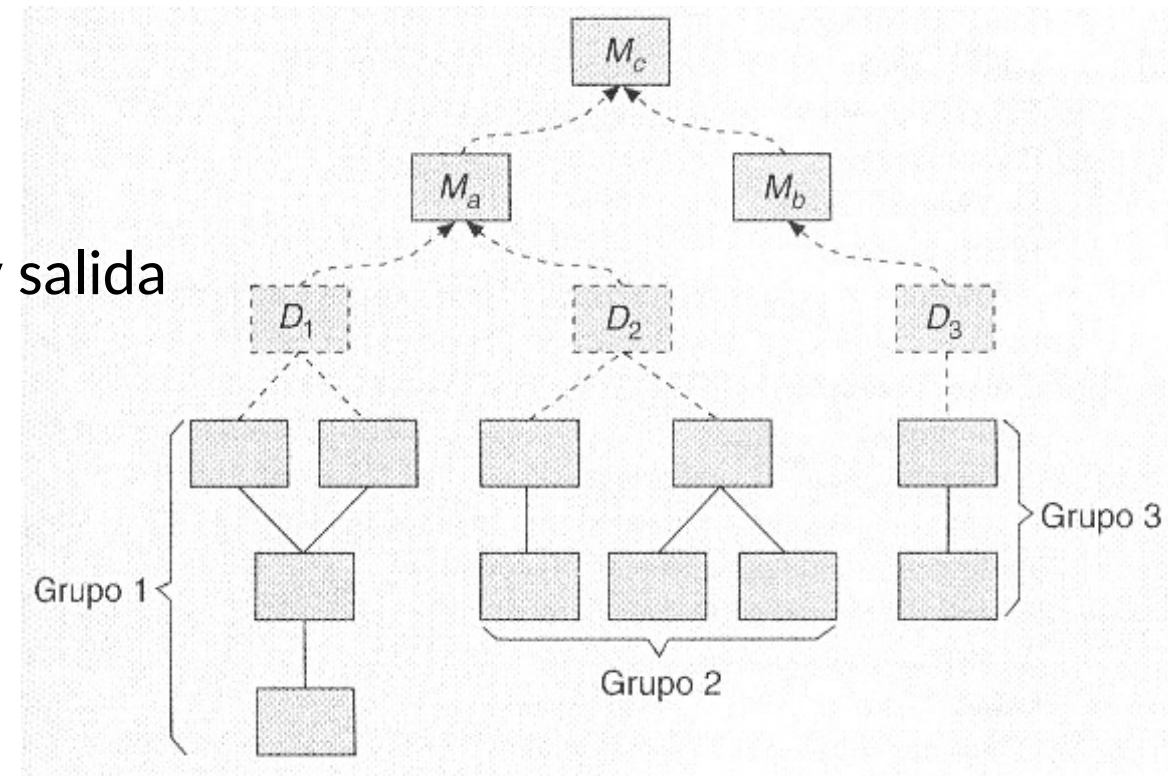
Dado que los módulos se integran de abajo hacia arriba, el proceso requerido de los módulos subordinados siempre está disponible y se elimina la necesidad de resguardos pero no así, los conductores.

Pruebas de Integración

❖ Integración ascendente

Pasos:

1. Combinar módulos de bajo nivel
2. Hacer conductor para coordinar entrada y salida
3. Probar el grupo
4. Eliminar conductores



Pruebas de Integración

- ❖ Hay discusión respecto a las ventajas y desventajas de los enfoques Ascendente con Descendente.
- ❖ La principal desventaja del enfoque Descendente es la necesidad de los resguardos.
- ❖ La principal desventaja de la opción Ascendente es que “el programa como entidad no existe hasta que se agrega el último módulo”.
- ❖ La elección depende de las características del software, un enfoque combinado (*prueba sándwich*) puede ser la mejor opción.

Pruebas de Integración

❖ Pruebas de regresión

- » Cada vez que se añade un nuevo módulo como parte de una prueba de integración, el software cambia.
- » Se establecen nuevos caminos, pueden ocurrir nuevas E/S y se invoca una nueva lógica de control.
- » Estos cambios pueden causar problemas con funciones que antes trabajaban perfectamente.
- » En el contexto de una estrategia de Prueba de integración, la Prueba de regresión implica volver a ejecutar un subconjunto de pruebas que se han llevado a cabo anteriormente para asegurarse de que los cambios no han propagado efectos colaterales no deseados.

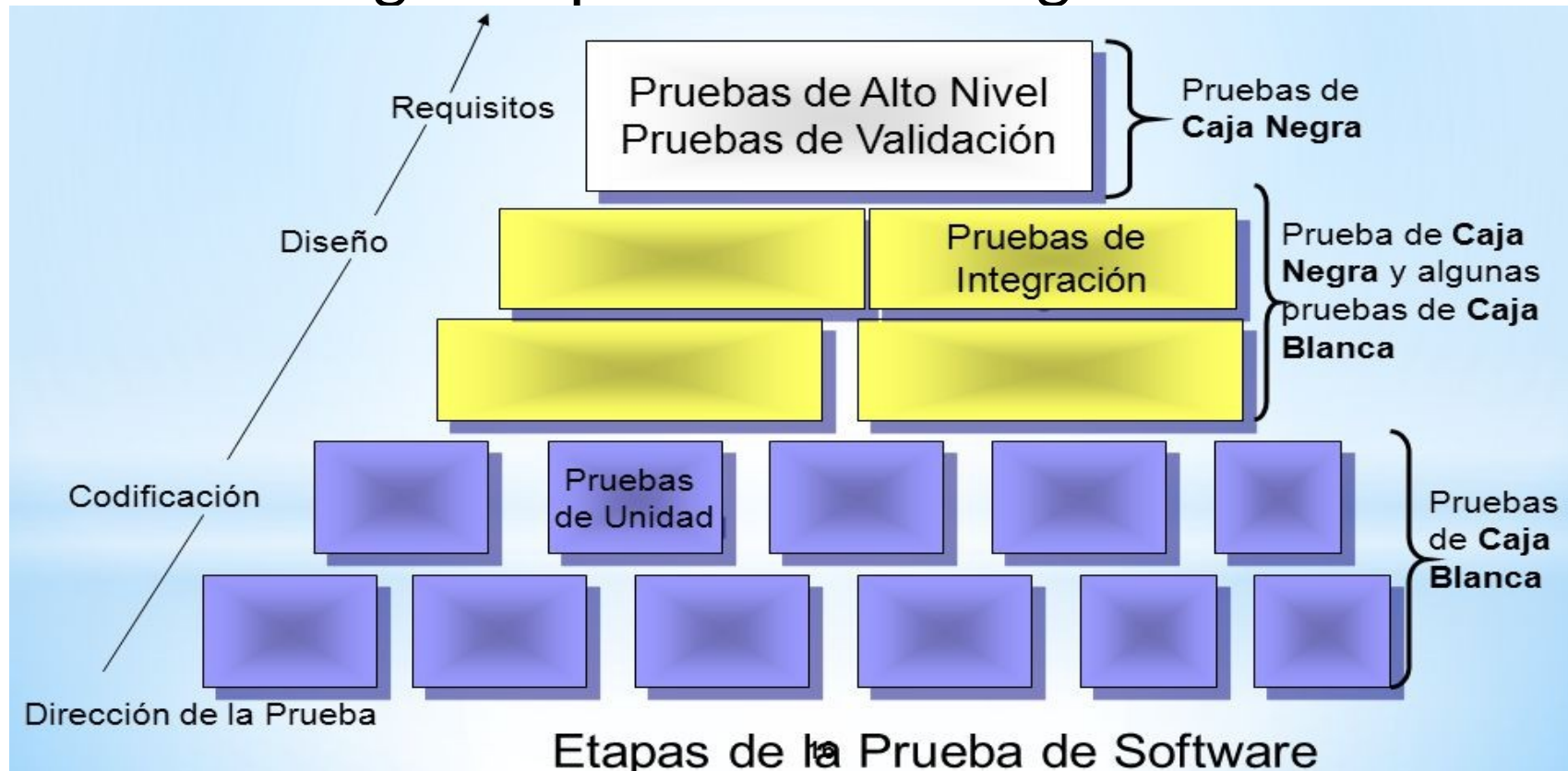
Pruebas de Integración

❖ Pruebas de regresión

- » Estas pruebas se pueden hacer manualmente, volviendo a realizar un subconjunto de todos los casos de prueba, o utilizando herramientas automáticas.
- » El conjunto de pruebas de regresión contiene tres clases diferentes de casos de prueba:
 - una muestra representativa de pruebas que ejercite todas las funciones del software.
 - pruebas adicionales que se centren en las funciones del software que son probablemente afectadas por el cambio.
 - pruebas que se centren en los componentes del software que han cambiado.

Pruebas de Integración

❖ Posible estrategia de pruebas de integración:



Pruebas de Unidad e Integración para software OO

❖ Prueba de Unidad:

- » Por lo general una clase encapsulada es el foco de la prueba de unidad.
- » Los métodos son las unidades comprobables más pequeñas.
- » La prueba de clase es el equivalente en este caso, la cual debe ser dirigida a las operaciones encapsuladas por la clase y el comportamiento de estado de ésta.

❖ Prueba de integración:

- » El software OO no tiene una estructura de control jerárquica obvia.
- » La prueba basada en hebra integra el conjunto de clases requeridas para responder a una entrada o evento.
- » La prueba basada en uso comienza con las clases independientes, luego las dependientes.

Pruebas de Validación

- ❖ La validación del software se consigue mediante una serie de pruebas que demuestren la conformidad con los requisitos.
- ❖ Una vez que se procede con cada caso de prueba de validación, puede darse una de las dos condiciones:
 1. *Las características de funcionamiento o de rendimiento están de acuerdo con las especificaciones y son aceptables.*
 2. *Se descubre una desviación de las especificaciones y se crea una lista de deficiencias.*

Pruebas de Validación

- ❖ Estas pruebas comienzan cuando finalizan las pruebas de integración.
- ❖ Revisión de la configuración: *Asegurar que todos los elementos de la configuración del software se hayan desarrollado apropiadamente, estén catalogados y contengan detalle suficiente para reforzar la fase de soporte.*

Pruebas de Validación

❖ Pruebas de aceptación

- » Las realiza el usuario final en lugar del responsable del desarrollo del sistema.
- » Una prueba de aceptación puede ir desde algo informal, hasta la ejecución sistemática de una serie de pruebas bien planificadas.
- » Dentro de las Pruebas de aceptación se pueden encontrar:
 - Pruebas ALFA: Las realizan los desarrolladores con clientes antes de liberar el producto.
 - Pruebas BETA: Se seleccionan los clientes que efectuarán la prueba. Los desarrolladores no se encuentran presentes.

Pruebas de Validación

❖ Pruebas de aceptación ALFA

- » Las pruebas alfa se hacen en un entorno controlado. Se llevan a cabo por un cliente en el lugar de desarrollo.
- » Se utiliza el software de forma natural con el desarrollador como observador del usuario, quien va registrando los errores y problemas de uso.
- » Se realizan después de que todos los procedimientos de prueba básicos, como las pruebas unitarias y pruebas de integración se han completado, y se produce después de las pruebas del sistema.
- » Esta no es la versión final del software. Pueden añadirse otras funcionalidades incluso después de la prueba alfa.

Pruebas de Validación

❖ Pruebas de aceptación BETA

- » Las pruebas beta se llevan a cabo por los usuarios finales del software en los lugares de trabajo de los clientes.
- » El desarrollador no está presente. La prueba beta es una aplicación en vivo del software en un entorno que no es controlado.
- » El cliente registra todos los problemas que encuentra durante la prueba beta e informa a intervalos regulares al desarrollador.
- » Las pruebas beta es la última fase de las fases de prueba y se hacen utilizando técnicas de caja negra.
- » A veces, la versión beta se lanza al mercado y, en función de las modificaciones realizadas, los comentarios de los usuarios o la ausencia de cambios en el software, se publica la versión final.

Pruebas de Validación

❖ Pruebas de aceptación BETA - Ejemplos

- » La explicación de cómo desarrollar una prueba beta de pre lanzamiento de una App para Apple Store se puede ver en:

<https://beta.apple.com/es-419>

- » Google APP para Android, programa de Beta testers

<https://support.google.com/websearch/answer/6307673?hl=es-419>

<https://www.google.com/android/beta?hl=es>

- » Otros:

<https://www.usertesting.com/>

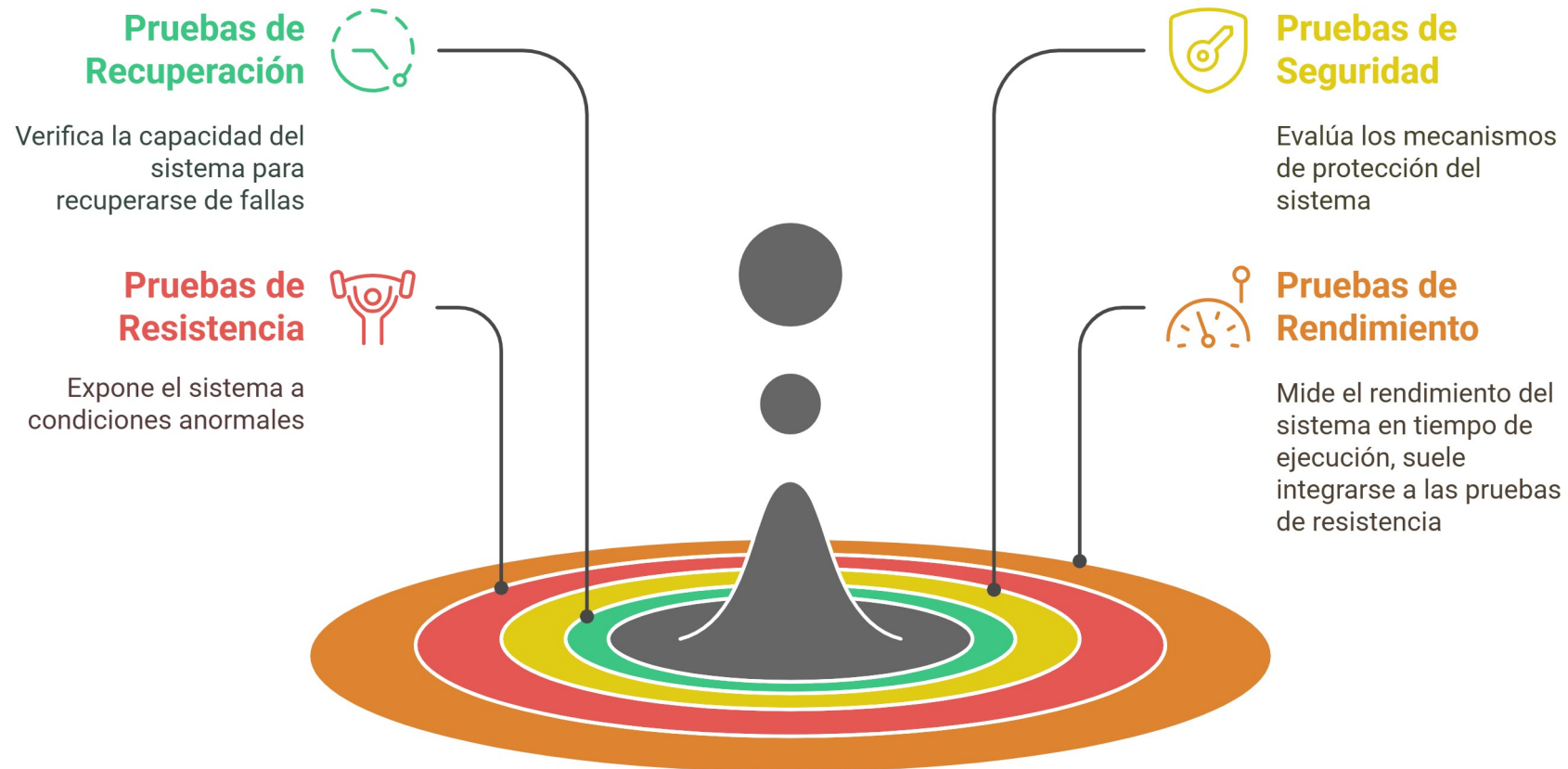
<https://www.utest.com/>

[Beta Testers: Únete a la comunidad de BetaTesting hoy](#)

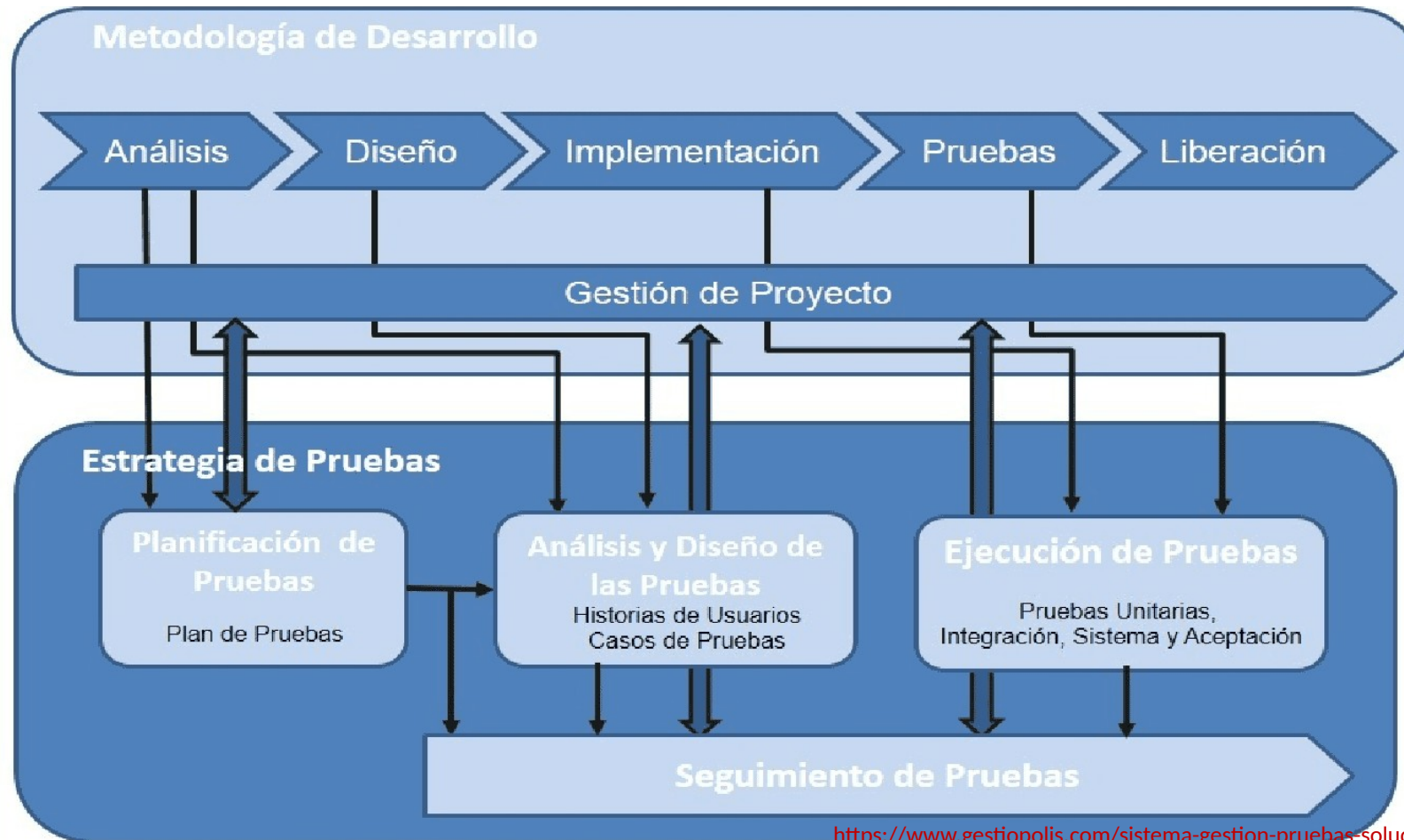
Pruebas del Sistema

- ❖ La prueba del sistema, está constituida por una serie de pruebas diferentes.
- ❖ Aunque cada prueba tiene un propósito diferente, todas trabajan para verificar que se han integrado adecuadamente todos los elementos del sistema y que realizan las funciones apropiadas.

Pruebas del Sistema



Una posible estrategia de pruebas



<https://www.gestiopolis.com/sistema-gestion-pruebas-soluciones-erp-enfocadas-paquetes-odoo-cuba/>

Pruebas de entornos especializados

- ❖ A medida que el software se hace más complejo, crece también la necesidad de enfoques de pruebas especializados.
 - » Pruebas de interfaces gráficas
 - » Pruebas de arquitecturas cliente-servidor
 - » Pruebas de la documentación y ayuda
 - » Pruebas de sistema en tiempo real

Pruebas de entornos especializados

❖ Pruebas de arquitecturas cliente-servidor

- » Pruebas de funcionalidad de la aplicación.
- » Prueba de servidor: *Probar las funciones de coordinación y manejo de datos del servidor. Desempeño del servidor (tiempo de respuesta y procesamiento total de los datos).*
- » Prueba de base de datos: *Probar la exactitud e integridad de los datos, examinar transacciones, asegurar que se almacenan, actualizan y recuperan los datos.*
- » Pruebas de transacciones: *Crear una serie de pruebas para asegurar que cada transacción se procese de acuerdo a los requisitos.*
- » Pruebas de comunicación de red: *Verificar comunicación entre los nodos, que el paso de mensajes, transacciones y tráfico de la red se realice sin errores.*

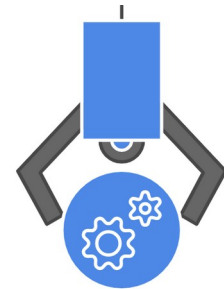
Pruebas de entornos especializados

- ❖ Pruebas de la documentación y funciones de ayuda
 - » Es importante para la aceptación del programa.
 - » Revisar la guía del usuario o funciones de ayuda en línea.
 - » La prueba de documentación se realiza en dos fases:
 1. Revisar e inspeccionar: examinar la claridad editorial del documento.
 2. Prueba en vivo: usar la documentación junto con el programa real.

Pruebas de entornos especializados

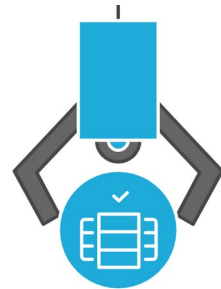
❖ Pruebas de sistemas de tiempo real

- » El diseño de los casos de prueba, además de los convencionales deben incluir manejo de eventos (interrupciones), temporización de los datos, el paralelismo entre las tareas, etc.



Pruebas de tareas

Probar las tareas de forma independiente, en búsqueda de errores lógicos



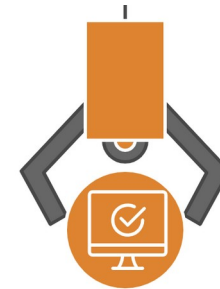
Pruebas de comportamiento

Simular el comportamiento del sistema de tiempo real y analizarlo como consecuencia de eventos.



Pruebas inter-tareas

Se prueban las tareas asincrónicas entre las cuales se sabe que hay comunicación

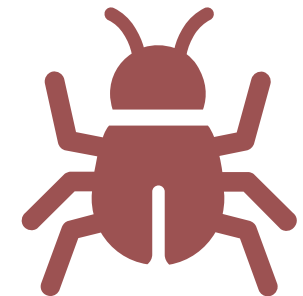


Pruebas de sistemas

Se prueba el software y hardware integrados.

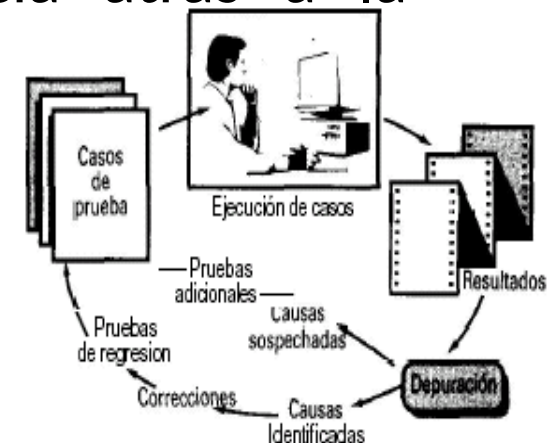
Depuración de programas

- ❖ La depuración de programas, es el proceso de identificar y corregir errores en programas informáticos.
- ❖ La depuración no es una prueba, pero siempre ocurre como consecuencia de la prueba efectiva.
 - » *Se descubre un error, la depuración elimina dicho error.*



El proceso de depuración

- ❖ El proceso de depuración siempre tiene uno de dos resultados:
1. Se encuentra la causa, se corrige y se elimina.
 2. No se encuentra la causa. La persona que realiza la depuración debe sospechar la causa, diseñar un caso de prueba que ayude a confirmar sus sospechas y el trabajo vuelve hacia atrás a la corrección del error de una forma iterativa.



El proceso de depuración

- ❖ Características de los errores:
 - » Síntoma lejano (geográficamente) de la causa
 - » Síntoma desaparece temporalmente al corregir otro error
 - » Síntoma producido por error
 - » Síntoma causado por error humano
 - » Síntoma causado por problemas de tiempo
 - » Condiciones de entrada difíciles de reproducir
 - » Síntoma intermitente (especialmente en desarrollos hardware-software)
 - » El síntoma se debe a causas distribuidas entre varias tareas que se ejecutan en diferentes procesadores

El proceso de depuración

❖ Enfoque:

